Koopman Operator Lifting via EML Observable Dictionaries: A PySINDy-Integrated Framework with CTF Benchmarking

Jim Beaver

April 2026 — Working Draft

Abstract

We introduce a framework for learning interpretable Koopman operator representations using the EML (Exp-Minus-Log) Sheffer operator $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ as the basis for observable dictionaries. Since every elementary function can be expressed as a binary tree of identical EML nodes, parameterized EML trees provide a *complete*, gradient-trainable search space for Koopman eigenfunctions. After Gumbel-softmax annealing and weight snapping, the learned observables are exact symbolic expressions — yielding closed-form Koopman eigenfunctions that black-box methods cannot produce. We replace transcendental function calls with Horner-form Taylor-series primitives, reducing the entire EML operator to fused multiply-add instructions for maximum GPU throughput. The resulting model is packaged as a PySINDy-compatible feature library and validated against the dynamicsai.org Common Task Framework (CTF) 12-metric benchmark on the Lorenz system, comparing to EDMD, deep Koopman autoencoders, and PySR baselines.

1 Introduction

The Koopman operator provides a linear (but infinite-dimensional) description of nonlinear dynamical systems [Koopman, 1931, Mezić, 2005, Budišić et al., 2012]. A central challenge is selecting the dictionary of observable functions that span a finite-dimensional invariant subspace. Traditional approaches — polynomials, radial basis functions, Fourier modes, delay coordinates — are chosen ad hoc and offer no completeness guarantee. Deep-learning methods [Takeishi et al., 2017, Lusch et al., 2018] learn flexible representations but sacrifice interpretability: the resulting observables are opaque neural networks with no closed-form expressions. Symbolic regression [Schmidt and Lipson, 2009, Cranmer, 2023] can discover closed-form expressions, but existing approaches lack the completeness guarantee that EML provides.

Odrzywołek [2026] discovered that the binary operator $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$, paired with the constant 1, generates *every* standard elementary function — the continuous-math analog of the NAND gate [Sheffer, 1913] for Boolean logic. This provides a natural, complete basis for Koopman observable dictionaries. Because every elementary function admits an EML tree representation, the search space is guaranteed to contain the true Koopman eigenfunctions whenever they are elementary — which holds for a wide class of physically-motivated systems (harmonic oscillators, Lotka–Volterra, pendula, etc.).

After training with Gumbel-softmax annealing [Jang et al., 2017], the continuous routing parameters snap to discrete one-hot selections, yielding *exact symbolic expressions*. The resulting Koopman eigenfunctions are linear combinations of these snapped EML observables and are therefore also elementary functions — fully interpretable and amenable to analytic manipulation.

Contributions.

- Parameterized EML trees as gradient-trainable Koopman observables with exact symbolic snap, providing a *complete* dictionary over elementary functions.
- GPU-optimized Taylor-series EML primitives in Horner form with range reduction, achieving zero transcendental function calls in both forward and backward passes.
- A fully vectorized tree architecture that evaluates all dictionary elements in a single batched call per tree level.
- A three-phase training pipeline (exploration, hardening, snap & fine-tune) with Gumbel-softmax annealing.
- PySINDy integration via `EMLLibrary`.
- CTF benchmark evaluation (E1–E12) on Lorenz, with comparison to EDMD, deep Koopman, and PySR baselines.

2 Background

2.1 The EML Operator

Odrzywołek [2026] introduced a binary operator that, together with the constant 1, generates every standard elementary function. The operator is defined as:

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \ln(y). \tag{1}$$

The grammar is context-free and trivially simple:

$$S \;\to\; 1 \;\mid\; \mathrm{eml}(S, S). \tag{2}$$

Every elementary expression therefore becomes a *binary tree of identical nodes.* Key identities include:

Function	EML Form	Depth
e^x	$\mathrm{eml}(x, 1)$	1
e	$\mathrm{eml}(1, 1)$	1
$\ln(x)$	$\mathrm{eml}\big(1,\ \mathrm{eml}(\mathrm{eml}(1, x),\ 1)\big)$	3
$-x$	depth-5 composition	5
$x \times y$	depth-8 composition	8

Internally, computations proceed over $\mathbb{C}$ via the principal branch of the complex logarithm. Trigonometric functions emerge through Euler's formula: $e^{i\varphi} = \cos\varphi + i\sin\varphi$, with $i = \sqrt{-1}$ obtainable from $\ln(-1) = i\pi$.

2.2 Koopman Operator Theory

Consider a discrete-time nonlinear dynamical system on a state space $\mathcal{M} \subseteq \mathbb{R}^n$:

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k). \tag{3}$$

The **Koopman operator** $\mathcal{K}$ acts on scalar-valued observables $g : \mathcal{M} \to \mathbb{C}$ by composition with the dynamics:

$$[\mathcal{K}\, g](\mathbf{x}) \;=\; g\big(F(\mathbf{x})\big). \tag{4}$$

This is an infinite-dimensional linear operator, even when F is nonlinear. The key insight: if we find a finite set of observables $\mathbf{g}(\mathbf{x}) = [g_1(\mathbf{x}), \ldots, g_N(\mathbf{x})]^\top$ that span a $\mathcal{K}$-invariant subspace, then the dynamics in lifted coordinates become:

$$\mathbf{g}(\mathbf{x}_{k+1}) \;=\; \mathbf{K}\, \mathbf{g}(\mathbf{x}_k), \tag{5}$$

where $\mathbf{K} \in \mathbb{C}^{N \times N}$ is a finite-dimensional **Koopman matrix**. The central problem is choosing the dictionary of observables $\{g_j\}$.

Extended Dynamic Mode Decomposition (EDMD) [Williams et al., 2015a] solves for $\mathbf{K}$ by least-squares regression given a fixed, hand-selected dictionary. The original DMD algorithm [Schmid, 2010, Tu et al., 2014] uses linear observables; EDMD extends this to nonlinear dictionaries. Typical choices — polynomials, radial basis functions, Fourier modes — are ad hoc and offer no guarantee that the relevant invariant subspace is spanned. Kernel EDMD [Williams et al., 2015b] and deep Koopman autoencoders [Lusch et al., 2018, Otto and Rowley, 2019] address expressiveness but sacrifice either tractability or interpretability. For a comprehensive review see Brunton et al. [2022].

2.3 SINDy and PySINDy

The Sparse Identification of Nonlinear Dynamics (SINDy) framework [Brunton et al., 2016] discovers governing equations by regressing time derivatives against a library of candidate nonlinear functions using sparsity-promoting optimization. PySINDy [de Silva et al., 2020, Kaptanoglu et al., 2022] provides a modular Python implementation with interchangeable feature libraries.

Our approach connects to SINDy through the shared notion of a feature library: rather than using pre-defined polynomial or Fourier features, we supply EML trees as the library. After symbolic snap, the discovered features can be fed into a standard SINDy regression pipeline, combining EML's completeness guarantee with SINDy's sparse model selection.

3 Method

3.1 EML Trees as Koopman Observables

3.1.1 The Master Formula

Each input to an EML node is parameterized as a soft selection among a candidate set:

$$\text{input}_i \;=\; \alpha_i + \beta_i \cdot x + \gamma_i \cdot f_{\text{child}}, \tag{6}$$

where $(\alpha_i, \beta_i, \gamma_i)$ are trainable weights that select among:

Selection	α_i	β_i	γ_i	Result
Constant 1	1	0	0	1
Input variable	0	1	0	x
Child output	0	0	1	f_{child}

During training, (α, β, γ) are treated as **Gumbel-softmax logits** [Jang et al., 2017] normalized to a simplex. Temperature τ anneals from soft (exploration) to hard (discrete). After convergence, the weights snap to one-hot $\{0, 1\}$, yielding an exact symbolic expression.

More generally, at each EML node i, both left and right inputs are selected from a candidate set $\mathcal{C}_i = \{1, x_0, x_1, \ldots, x_{n-1}\}$ (for leaves) or $\mathcal{C}_i = \{1, x_0, \ldots, x_{n-1}, f_{\text{child}}\}$ (for interior nodes). The selection is parameterized by logits $\ell_i \in \mathbb{R}^{|\mathcal{C}_i|}$:

$$w_i^{(j)} = \frac{\exp\big((\ell_i^{(j)} + G_j)/\tau\big)}{\sum_k \exp\big((\ell_i^{(k)} + G_k)/\tau\big)}, \qquad G_j \sim \text{Gumbel}(0, 1). \tag{7}$$

The selected input is $\sum_j w_i^{(j)} c_j$ where $c_j \in \mathcal{C}_i$. As $\tau \to 0$, this converges to hard one-hot selection.

3.1.2 Full Level-2 Example

The complete level-2 master formula (14 parameters for univariate x) is:

$$\begin{aligned} F(x) = \text{eml}\Big[\; & \alpha_1 + \beta_1 x + \gamma_1 \text{ eml}(\alpha_3 + \beta_3 x,\ \alpha_4 + \beta_4 x), \\ & \alpha_2 + \beta_2 x + \gamma_2 \text{ eml}(\alpha_5 + \beta_5 x,\ \alpha_6 + \beta_6 x) \;\Big]. \end{aligned} \tag{8}$$

Setting $\alpha_1 = 0, \beta_1 = 1, \gamma_1 = 0$ and $\alpha_2 = 1, \beta_2 = \gamma_2 = 0$ recovers $\exp(x)$. The general depth-d formula has $5 \cdot 2^d - 6$ parameters.

3.1.3 Koopman Dictionary Construction

We construct N independent EML trees, each of depth d, to form the observable dictionary:

$$\mathbf{g}_\theta(\mathbf{x}) \;=\; \big[\, T_1^\theta(\mathbf{x}),\ T_2^\theta(\mathbf{x}),\ \ldots,\ T_N^\theta(\mathbf{x}) \,\big]^\top, \tag{9}$$

where T_j^θ denotes the j-th EML tree with trainable routing parameters θ. The full Koopman model is then:

$$\mathbf{g}_\theta(\mathbf{x}_{k+1}) \;\approx\; \mathbf{K}\, \mathbf{g}_\theta(\mathbf{x}_k), \qquad \mathbf{x}_k \;\approx\; \mathbf{C}\, \mathbf{g}_\theta(\mathbf{x}_k), \tag{10}$$

where $\mathbf{K} \in \mathbb{R}^{N \times N}$ is the Koopman matrix and $\mathbf{C} \in \mathbb{R}^{n \times N}$ is the decoder (reconstruction map).

3.1.4 Vectorized Tree Architecture

All N trees are packed into contiguous parameter tensors, one per tree level:

$$\text{level_logits}[l] \;\in\; \mathbb{R}^{N \times n_l \times |\mathcal{C}_l| \times 2}, \tag{11}$$

where $n_l = 2^{d-1-l}$ is the number of nodes at level l and the trailing dimension 2 indexes left/right inputs. Bottom-up evaluation proceeds as d sequential batched operations (Algorithm 1).

Algorithm 1 Vectorized EML Tree Evaluation

Require: State batch $\mathbf{x} \in \mathbb{R}^{B\times n}$, tree depth d, temperature τ
Ensure: Lifted observables $\mathbf{g} \in \mathbb{R}^{B\times N}$

```
for l = 0, 1, ..., d − 1 (bottom to top) do
    Compute Gumbel-softmax weights W ∈ R^{B×N×n_l×|C_l|×2}
    Assemble candidate tensors (const 1, input vars, child outputs from level l−1)
    Weighted selection: eml_x, eml_y ∈ R^{B×N×n_l}
    Clamp inputs: eml_x ← clamp(eml_x, −C, C)
    Batched EML: out_l = eml_Taylor(eml_x, eml_y)
end for
return out_{d−1} ∈ R^{B×N} (root outputs = lifted state)
```

3.1.5 Parameter Counts

The model is compact by modern standards:

Depth d	Params/tree	$N=16$	$N=32$	Expressible functions
2	14	224	448	e^x, $\ln x$, e^{e^x}
3	34	544	1,088	$-x$, $1/x$, most unary
4	74	1,184	2,368	$x+y$, $x\cdot y$, trig
5	154	2,464	4,928	Full calculator

Odrzywołek [2026] reports ∼25% blind recovery at depth 3–4, improving to 100% with warm-start initialization.

3.2 Taylor-Series GPU Primitives

Both exp and ln can be replaced by polynomial approximations in Horner form, reducing the entire EML operator to fused multiply-add (FMA) instructions with zero transcendental function calls. This is critical for GPU throughput, especially with nested tree compositions where each node calls EML.

3.2.1 Taylor Exponential with Range Reduction

Decompose $x = k\ln 2 + r$ where $k = \lfloor x/\ln 2 \rceil$ and $|r| \le \frac{\ln 2}{2} \approx 0.347$. Then:

$$\exp(x) = 2^k \cdot \exp(r), \qquad \exp(r) = \sum_{n=0}^{M} \frac{r^n}{n!}. \tag{12}$$

The Taylor series is evaluated in Horner form (innermost term outward):

$$\exp(r) \;=\; 1 + r\Big(1 + \frac{r}{2}\Big(1 + \frac{r}{3}\Big(1 + \cdots \frac{r}{M}\Big)\Big)\Big)\,. \tag{13}$$

Each step is one multiply-add. With $|r| \le 0.347$, order $M = 12$ gives error $\lesssim 10^{-16}$ (double precision). The 2^k scaling is a single `ldexp`-equivalent operation.

3.2.2 Taylor Logarithm via Arctanh

Range-reduce: $y = m \cdot 2^e$ with $m \in [0.5, 1.0)$, so $\ln(y) = e \ln 2 + \ln(m)$. Then use the arctanh series, which converges much faster than the naive $\ln(1+t)$ expansion:

$$\ln(m) \;=\; 2 \operatorname{arctanh}\left(\frac{m-1}{m+1}\right) \;=\; 2 \sum_{k=0}^{M} \frac{u^{2k+1}}{2k+1}, \quad u = \frac{m-1}{m+1}. \tag{14}$$

For $m \in [0.5, 1)$ we have $|u| \leq \frac{1}{3}$, giving rapid convergence. Horner evaluation:

$$\sum_{k=0}^{M} \frac{u^{2k+1}}{2k+1} \;=\; u\left(\frac{1}{1} + u^2\left(\frac{1}{3} + u^2\left(\frac{1}{5} + \cdots\right)\right)\right). \tag{15}$$

Order $M = 16$ gives machine-epsilon accuracy for ln.

3.2.3 Combined EML Primitive

The GPU-optimized EML primitive is:

$$\mathrm{eml}_{\mathrm{Taylor}}(x, y) \;=\; \mathrm{horner_exp}(x, M_e) \;-\; \mathrm{horner_ln}(y, M_l), \tag{16}$$

with gradients $\frac{\partial}{\partial x} \mathrm{eml} = \exp(x)$ and $\frac{\partial}{\partial y} \mathrm{eml} = -\frac{1}{y}$, both computable via the same polynomial machinery. A custom `torch.autograd.Function` [Paszke et al., 2019] avoids redundant recomputation in the backward pass: the forward stores x and y and reuses the Horner evaluation for gradient computation.

For a depth-d tree with N parallel observables evaluated over a batch of size B, the forward pass requires d sequential EML calls, each operating on tensors of shape $[B, N, 2^{d-1-l}]$ at level l. All nodes at the same depth are computed in a single batched call — no Python loops over individual nodes.

3.3 PySINDy Integration

We package the EML dictionary as a PySINDy-compatible feature library, `EMLLibrary`, supporting two modes:

- **Enumerative mode:** Generate all distinct EML trees up to a specified depth (filtered by algebraic equivalence). This provides a finite but complete basis for SINDy regression at a chosen complexity level.
- **Pre-trained mode:** Supply trees already trained via Gumbel-softmax Koopman optimization (Section 4.1). After snapping, each tree is a fixed symbolic feature. SINDy's sparsity-promoting regression then selects the most relevant subset.

Both modes implement the standard PySINDy `BaseFeatureLibrary` interface, allowing seamless composition with existing PySINDy optimizers (STLSQ, SR3, etc.).

4 Experimental Setup

4.1 Training Pipeline

4.1.1 Loss Function

Given trajectory data $\{(\mathbf{x}_k, \mathbf{x}_{k+1})\}_{k=1}^{K}$, the joint objective is:

$$\mathcal{L}(\theta, \mathbf{K}, \mathbf{C}) \;=\; \underbrace{\sum_k \left\| \mathbf{g}_\theta(\mathbf{x}_{k+1}) - \mathbf{K}\,\mathbf{g}_\theta(\mathbf{x}_k) \right\|^2}_{\text{prediction}} + \lambda \underbrace{\sum_k \left\| \mathbf{x}_k - \mathbf{C}\,\mathbf{g}_\theta(\mathbf{x}_k) \right\|^2}_{\text{reconstruction}} + \mu \underbrace{\|\mathbf{K}\|_F^2}_{\text{regularization}} \quad . \tag{17}$$

All three components are differentiable through the EML trees (the Taylor polynomials have clean analytic gradients).

4.1.2 Three-Phase Annealing Schedule

Training proceeds in three phases:

1. **Phase 1 — Exploration** (epochs 0 to $0.6T$): Anneal τ from $\tau_{\max}$ (e.g., 2.0) to 1.0. All parameters $(\theta, \mathbf{K}, \mathbf{C})$ are trainable. Gumbel noise encourages exploration of different routing configurations.

2. **Phase 2 — Hardening** (epochs $0.6T$ to $0.9T$): Anneal τ from 1.0 to $\tau_{\min}$ (e.g., 0.1). Routing weights sharpen toward discrete selections; the trees commit to specific functional forms.

3. **Phase 3 — Snap & Fine-Tune** (epochs $0.9T$ to T): Snap all routing logits to hard one-hot (argmax). Freeze tree parameters θ. Fine-tune only $\mathbf{K}$ and $\mathbf{C}$ at reduced learning rate.

Overflow hazard. Nested EML compositions produce iterated exponentials ($e^{e^{e^x}}$) during exploration. EML inputs are clamped to $[-C, C]$ with $C = 5$ (so $\exp(C) \approx 148$), and state data is normalized to zero-mean unit-variance before lifting. The tight clamp is essential: $C = 20$ yields $\exp(20) \approx 5 \times 10^8$, causing catastrophic loss magnitudes in early training.

4.1.3 Warm-Start Strategies

Convergence from random initialization degrades sharply beyond depth 4. Two warm-start strategies improve recovery:

1. **EDMD bootstrap:** Run standard EDMD with a polynomial dictionary first. Initialize EML tree routing to mimic the best polynomial observables (since polynomials are elementary, they have EML representations).

2. **Curriculum depth:** Train at depth 2 until convergence, then grow each tree by one level (appending new leaf nodes with random logits) and continue training. The existing structure provides a strong initialization for the expanded tree.

4.2 CTF Benchmark Framework

We evaluate on the dynamicsai.org Common Task Framework (CTF) [Wyder et al., 2025], which defines 12 standardized metrics (E1–E12) for dynamical-system learning. Each metric follows the scoring formula $E = 100 \times (1 - \|\text{pred} - \text{truth}\|/\|\text{truth}\|)$, where $E = 100$ is perfect and $E < 0$ is worse than trivial. For long-term metrics the comparison is made in log-power-spectrum space. The 12 metrics are grouped into four categories:

- **Forecasting (E1–E2):** E1 evaluates short-term trajectory tracking (normalized RMSE over 1–5 Lyapunov times). E2 evaluates long-term spectral fidelity (log power spectrum error).
- **Noisy data (E3–E6):** E3/E4 test reconstruction and forecasting from medium-noise observations (SNR $\approx 20\,\text{dB}$). E5/E6 repeat at high noise (SNR $\approx 10\,\text{dB}$).
- **Limited data (E7–E10):** E7/E8 evaluate short- and long-term performance when trained on only 10% of the clean data. E9/E10 repeat with 10% noisy data.
- **Parametric generalization (E11–E12):** E11 tests interpolation: train at parameter values bracketing the test value (e.g., $\rho \in \{26, 30\}$, test at $\rho = 28$). E12 tests extrapolation: train at $\rho \in \{26, 28\}$, test at $\rho = 32$.

We use the Lorenz-63 system ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$) as the primary test case, with 20,000 training pairs sampled at $\Delta t = 0.01$ from 20 trajectories normalized to zero-mean unit-variance.

4.3 Baselines

We compare four methods:

1. **EML-Koopman (ours):** $N = 16$ trees at depth 2 (720 parameters), three-phase training with $T = 1200$ epochs, batch size 2048, input clamping at ± 5. Taylor orders: $M_e = 10$, $M_l = 12$.
2. **EDMD-Poly:** Polynomial dictionary up to degree 4 ($\binom{n+4}{4} = 35$ features for $n = 3$), least-squares $\mathbf{K}$.
3. **EDMD-RBF:** 200 Gaussian RBF centers selected randomly from the training data, $\sigma = 1.0$.
4. **Deep Koopman:** 3-layer MLP autoencoder ($3 \to 128 \to 128 \to 32$), symmetric decoder, linear dynamics in 32-dimensional latent space (43,299 parameters), trained for 1200 epochs.

5 Results

5.1 CTF Lorenz Benchmark

Table 1 summarizes the CTF Lorenz benchmark results for all methods. All models were trained on 20,000 state pairs from 20 Lorenz trajectories ($\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $\Delta t = 0.01$), normalized to zero-mean unit-variance. Forecasting is evaluated on a held-out trajectory segment (501 time steps).

Table 1: CTF Lorenz benchmark comparison. E1 = short-term forecasting score ($100 \times (1 - \text{NRMSE})$, higher is better). E2 = long-term spectral score (higher is better). Valid steps = number of time steps before prediction error exceeds 0.5σ of the test trajectory. Published CTF leaderboard values (E1–E12 average) shown for context.

Method	E1	E2	RMSE	Valid Steps	Params
EML-Koopman (ours)	**7.70**	−12.23	**0.888**	**146**	720
EDMD-Poly (degree 4)	−1.11	**52.94**	0.973	84	∼35
EDMD-RBF (200 centers)	−10,216	−252	99.3	0	∼200
Deep Koopman (autoencoder)	0.63	−34.28	0.956	105	43,299
LSTM (CTF published)	avg = 64.54		—	—	—
DeepONet (CTF published)	avg = 57.80		—	—	—
Reservoir (CTF published)	avg = 54.87		—	—	—
KAN (CTF published)	avg = 47.28		—	—	—
SINDy (CTF published)	avg = −3.00		—	—	—
PyKoopman (CTF published)	avg = −20.11		—	—	—

Short-term forecasting (E1). EML-Koopman achieves E1 = 7.70, the highest among all implemented baselines and substantially above the two other interpretable methods on the CTF leaderboard: SINDy (−3.00) and PyKoopman (−20.11). The valid prediction horizon of 146 time steps (1.46 Lyapunov times at $\Delta t = 0.01$) exceeds EDMD-Poly (84 steps) and Deep Koopman (105 steps) despite using 60× fewer parameters than the autoencoder. The gap to black-box methods (LSTM 64.54, DeepONet 57.80) reflects the expected trade-off between parameter capacity and interpretability.

Long-term statistics (E2). EDMD-Poly achieves the highest E2 (52.94) by a wide margin, because its polynomial basis functions naturally capture the power spectral shape of the Lorenz attractor even when short-term prediction diverges. EML-Koopman's E2 of −12.23 indicates room for improvement in long-term spectral fidelity. A systematic depth sweep (Section 5.3) investigates whether deeper EML trees can close this gap.

Failure modes. EDMD-RBF diverges catastrophically (E1 = −10,216) because random Gaussian centers in the normalized 3D Lorenz state space produce a severely ill-conditioned Koopman matrix. This underscores the importance of dictionary design — a problem that EML addresses through gradient-based observable discovery.

5.2 Discovered Symbolic Observables

After Phase 3 snap, all 16 EML tree observables reduce to closed-form elementary functions. Representative examples (with x_0, x_1, x_2 corresponding to normalized Lorenz x, y, z):

Observable	Snapped Formula
g_0	$\mathrm{eml}(\mathrm{eml}(1, x_1),\ x_1)$
g_1	$\mathrm{eml}(\mathrm{eml}(1, x_1),\ 1)$
g_2	$\mathrm{eml}(\mathrm{eml}(1, x_0),\ 1)$
g_4	$\mathrm{eml}(\mathrm{eml}(1, x_1),\ x_2)$
g_{10}	$\mathrm{eml}(\mathrm{eml}(1, x_0),\ x_1)$

These simplify to:

$$
\begin{aligned}
g_1(\mathbf{x}) &= \exp(e - \ln x_1) = e \cdot x_1^{-1} \cdot \exp(0) \quad \text{(involves } 1/y\text{)}, \\
g_2(\mathbf{x}) &= \exp(e - \ln x_0) \quad \text{(involves } 1/x\text{)}, \\
g_4(\mathbf{x}) &= \exp(e - \ln x_1) - \ln x_2 \quad \text{(mixes exponential and logarithmic)}.
\end{aligned}
$$

The model predominantly discovers observables built from $\mathrm{eml}(1, x_i) = e - \ln x_i$ as an inner expression, then applies a second EML layer. This structure is natural: depth-2 trees can represent e^x, $\ln x$, and their compositions, but not yet multiplication or addition (which require depth ≥ 5). The abundance of logarithmic and exponential-reciprocal features suggests the model has found the most informative nonlinear transformations available at depth 2.

Interpretability advantage. Unlike the 43,299-parameter Deep Koopman autoencoder — whose observables are opaque 3-layer neural networks — every EML observable is a human-readable formula. The resulting Koopman eigenfunctions $\varphi_j(\mathbf{x}) = \mathbf{v}_j^\top \mathbf{g}(\mathbf{x})$ are therefore explicit linear combinations of elementary functions, amenable to analytic simplification and physical interpretation.

5.3 Progressive Depth Analysis

A key hypothesis motivating this work is that deeper EML trees — which can express a richer class of elementary functions — should improve Koopman approximation quality. To test this, we swept tree depths $d \in \{2, 3, 4, 5\}$ with multiple observable counts $N \in \{16, 24, 32\}$ per depth, training each configuration from random initialization with depth-adapted hyperparameters (tighter input clamping and lower learning rates at greater depths to manage numerical instability from iterated exponentials). Table 2 summarizes all 11 configurations.

Deeper trees do not improve E2. Contrary to the initial hypothesis, the best E2 (-12.23) comes from the *shallowest* configuration (d2-N16, 720 parameters). Deeper trees consistently produce worse long-term spectral fidelity, with E2 ranging from -14.22 (d5-N16) to -70.24 (d2-N32). Similarly, short-term E1 degrades from 7.70 at depth 2 to as low as 1.86 at depth 3, indicating that the optimization landscape becomes significantly harder at greater depths.

Formula collapse at depth 3+. Inspection of the snapped formulas reveals a structural explanation. Table 3 shows representative observables from each depth. At depth 2, the optimizer discovers diverse two-layer compositions such as $\mathrm{eml}(\mathrm{eml}(1, x_1), x_0)$ and $\mathrm{eml}(\mathrm{eml}(1, x_0), x_2)$ that mix multiple state variables through exponential and logarithmic transformations. At depth 3 and beyond, most trees collapse to shallow forms — typically $\mathrm{eml}(1, x_i) = e - \ln x_i$, using only the root node while the deeper structure goes unused. This collapse suggests that gradient signal attenuates through nested clamped exponentials, preventing the optimizer from coordinating routing decisions across multiple tree levels.

Table 2: Progressive depth sweep on CTF Lorenz. Each row is a separate training run from random initialization. Clamp = symmetric input clamp $[-C, C]$ applied before each EML call. Best E1 and best E2 among EML configurations are bolded.

Config	d	N	Clamp	Epochs	E1	E2	RMSE	Valid Steps	Params
d2-N16	2	16	5.0	1200	**7.70**	**−12.23**	**0.888**	**146**	720
d2-N24	2	24	5.0	1500	7.84	−47.40	0.887	146	1,272
d2-N32	2	32	5.0	1500	7.20	−70.24	0.893	144	1,952
d3-N16	3	16	4.0	1500	5.32	−48.58	0.911	143	1,296
d3-N24	3	24	4.0	2000	4.08	−37.29	0.923	140	2,136
d3-N32	3	32	4.0	2000	1.86	−39.15	0.944	134	3,104
d4-N16	4	16	3.0	2000	2.09	−68.88	0.942	134	2,448
d4-N24	4	24	3.0	2500	4.56	−24.67	0.918	141	3,864
d4-N32	4	32	3.0	2500	3.43	−27.71	0.929	140	5,408
d5-N16	5	16	2.5	2500	2.35	−14.22	0.940	136	4,752
d5-N24	5	24	2.5	3000	4.53	−22.82	0.919	144	7,320
EDMD-Poly (reference)					−1.11	52.94	0.973	84	∼35

Structural barrier to E2. The persistent 65-point E2 gap between EML-Koopman (−12.23) and EDMD-Poly (52.94) has a structural explanation. EDMD-Poly's 4th-degree polynomial dictionary directly contains monomials (x^2, xy, x^2z, etc.) that closely approximate the Lorenz attractor's power spectral shape. In EML tree representation, *addition* requires depth ≥ 5 and *multiplication* requires depth ≥ 8 [Odrzywołek, 2026]. The depth-2 trees used in our best configuration cannot express any polynomial of degree ≥ 2; their observable space is restricted to compositions of exp and ln — functions whose spectral signatures differ fundamentally from the polynomial features that capture Lorenz long-term statistics. Closing this gap requires either much deeper trees (with correspondingly harder optimization) or a hybrid dictionary combining EML and polynomial features.

Fewer observables yield better E2. Within each depth, the smallest dictionary ($N = 16$) consistently achieves the best or near-best E2. This suggests that over-parameterized dictionaries introduce redundant or near-degenerate observables whose spectral contributions interfere destructively in the long-term regime.

5.4 Full CTF E1–E12 Evaluation

We evaluate EML-Koopman (d2-N16, 720 parameters) across all 12 CTF metrics by constructing the required data scenarios: noisy observations at 20 dB and 10 dB SNR (E3–E6), 10% training data subsets (E7–E10), and parametric variations of the Lorenz ρ parameter (E11–E12). For noisy-data metrics, the model is trained on clean data and the Koopman lift→reconstruct pipeline acts as a learned denoiser; for limited-data and parametric metrics, separate models are trained on the reduced or multi-parameter datasets. Table 4 reports all 12 scores.

Denoising via Koopman lifting (E3, E5). The most striking result is the strong performance on noisy-data reconstruction: E3 = 33.68 at 20 dB SNR and E5 = 30.65 at 10 dB SNR. The EML

Table 3: Representative snapped EML formulas at each tree depth, illustrating the progressive collapse of deeper trees toward simple root-only expressions. Variables x_0, x_1, x_2 correspond to normalized Lorenz x, y, z.

Depth	Observable	Snapped Formula
2	g_0	$\mathrm{eml}\big(\mathrm{eml}(1, x_1),\ \mathrm{eml}(1, x_1)\big)$
	g_1	$\mathrm{eml}\big(\mathrm{eml}(1, x_1),\ x_1\big)$
	g_3	$\mathrm{eml}\big(\mathrm{eml}(1, x_1),\ x_0\big)$
	g_6	$\mathrm{eml}\big(\mathrm{eml}(1, x_0),\ x_2\big)$
3	g_0	$\mathrm{eml}(1,\ x_0)$
	g_1	$\mathrm{eml}(1,\ x_1)$
	g_6	$\mathrm{eml}(1,\ x_2)$
4	g_0	$\mathrm{eml}(1,\ x_0)$
	g_3	$\mathrm{eml}(x_1,\ x_1)$
	g_6	$\mathrm{eml}(x_1,\ x_0)$
5	g_0	$\mathrm{eml}(1,\ x_1)$
	g_1	$\mathrm{eml}(x_2,\ x_0)$
	g_2	$\mathrm{eml}(x_1,\ x_0)$

observable dictionary spans a low-dimensional manifold (16 observables in 3D state space); lifting noisy observations into this manifold and decoding back to state space naturally rejects noise that lies outside the learned subspace. This is analogous to denoising via projection onto a low-rank subspace, but the nonlinear EML lifting provides a richer manifold geometry than linear PCA. The modest degradation from 20 dB to 10 dB (3 points) indicates robust denoising even under severe noise contamination.

Robustness under data scarcity (E7). Training on only 10% of the clean data yields E7 = 7.77, which *exceeds* the full-data E1 = 6.38. This counter-intuitive result reflects the strong inductive bias of the 720-parameter model: with fewer training pairs, the model is naturally regularized and avoids overfitting to the training trajectories. The E8 long-term score (-47.37) remains negative, consistent with the structural spectral limitation discussed above.

Noisy limited-data regime (E9–E10). Under simultaneous noise and data scarcity, EML-Koopman still produces positive short-term scores (E9 = 3.43) and avoids catastrophic failure in the long-term metric (E10 = -0.18, near zero rather than deeply negative). This graceful degradation is attributable to the compact model size: with only 720 parameters, there is insufficient capacity to overfit noisy training data.

Parametric generalization (E11–E12). Interpolation (E11 = 1.16) is marginally positive and extrapolation (E12 = -0.12) is near zero. The learned Koopman operator $\mathbf{K}$ encodes dynamics at specific parameter values and does not transfer across the bifurcation parameter ρ without explicit parameterization. Addressing this limitation would require parameter-conditioned architectures, e.g., making the Koopman matrix a function of ρ: $\mathbf{K}(\rho)$.

Table 4: Full CTF E1–E12 scorecard for EML-Koopman (depth 2, $N = 16$, 720 parameters) on the Lorenz-63 system. Scores follow the CTF formula: $E = 100 \times (1 - \|\text{pred} - \text{truth}\|/\|\text{truth}\|)$. Long-term metrics (E2, E4, E6, E8, E10) are computed in log-power-spectrum space. Published CTF leaderboard averages are shown for comparison.

Metric	Category	Score
E1	Short-term forecasting	6.38
E2	Long-term spectral	3.77
E3	Medium-noise reconstruction	33.68
E4	Medium-noise long-term forecast	−41.87
E5	High-noise reconstruction	30.65
E6	High-noise long-term forecast	−41.84
E7	Limited data, short-term	7.77
E8	Limited data, long-term	−47.37
E9	Limited noisy data, short-term	3.43
E10	Limited noisy data, long-term	−0.18
E11	Parametric interpolation	1.16
E12	Parametric extrapolation	−0.12
	Average (E1–E12)	**−3.71**

CTF Published (avg)	Avg Score
LSTM	64.54
DeepONet	57.80
Reservoir	54.87
KAN	47.28
ODE-LSTM	41.67
SINDy	−3.00
EML-Koopman (ours)	**−3.71**
PyKoopman	−20.11

Long-term spectral metrics. E4, E6, and E8 are all significantly negative (−41 to −47), consistent with the E2 analysis in Section 5.3: the exp/ln observables at depth 2 cannot reproduce the Lorenz power spectrum shape that polynomial bases capture naturally.

Overall positioning. The E1–E12 average of −3.71 places EML-Koopman within one point of SINDy (−3.00) and 16 points above PyKoopman (−20.11) on the CTF leaderboard — competitive with the best interpretable methods while providing closed-form symbolic observables. The gap to black-box methods (LSTM at 64.54, DeepONet at 57.80) reflects the fundamental trade-off between the 720-parameter symbolic model and neural networks with 10K–100K+ parameters. Notably, EML-Koopman's denoising scores (E3, E5) represent a distinctive capability not typically associated with Koopman methods, suggesting that the nonlinear manifold structure of EML dictionaries has practical value beyond forecasting accuracy.

6 Discussion

6.1 Eigendecomposition and Interpretability

After training and snapping, the explicit Koopman matrix $\mathbf{K}$ admits eigendecomposition:

$$\mathbf{K} = \mathbf{V}\,\Lambda\,\mathbf{V}^{-1}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_N). \tag{18}$$

This yields:

- **Koopman eigenvalues** λ_j: encode frequencies ($\omega_j = \mathrm{Im}(\ln \lambda_j)$) and growth/decay rates ($\sigma_j = \mathrm{Re}(\ln \lambda_j)$).
- **Koopman eigenfunctions:** $\varphi_j(\mathbf{x}) = \mathbf{v}_j^\top \mathbf{g}_\theta(\mathbf{x})$, i.e., linear combinations of the snapped EML observables. Since each g_i is a closed-form elementary expression, φ_j is also elementary — fully interpretable.
- **Koopman modes:** $\mathbf{m}_j = \mathbf{C}\,\mathbf{v}_j$, linking each eigenfunction to its contribution in state space.

This is a fundamental advantage over deep-learning Koopman approaches [e.g., LRAN, Otto and Rowley, 2019, Lusch et al., 2018]: instead of opaque neural network observables, we get symbolic formulas that can be analyzed, simplified, and compared to physics.

6.2 GPU Acceleration and Backend Comparison

The EML tree architecture is naturally parallelizable: all N trees at each level are evaluated in a single batched call, and the Horner-form Taylor primitives decompose into multiply-add chains. We benchmark four computational backends on the Lorenz-63 task (depth 2, $N = 16$, 20,000 training pairs, batch size 2048) to measure the realized speedup from GPU acceleration.

6.2.1 Backends

1. **CPU Taylor** — Horner-form Taylor series on CPU (the baseline from all previous experiments).
2. **GPU Taylor** — The same Taylor implementation moved to GPU via `.to("cuda")`. Each Horner loop step dispatches a separate CUDA kernel.
3. **GPU Native** — Replace the Taylor series with `torch.exp`/`torch.log` (CUDA's built-in math library, which uses the same range-reduction and minimax polynomial techniques as SLEEF [Shibata, 2025]).
4. **GPU Compiled** — Apply `torch.compile` to fuse the exp-minus-log into a single Triton kernel, eliminating intermediate memory round-trips.

6.2.2 Related Libraries

No native CUDA implementation of the full EML operator exists at the time of writing. The closest related work includes: SLEEF (`https://sleef.org/cuda.xhtml`), which provides inlinable CUDA device functions for exp and ln with configurable ULP error bounds (0.5–3.5 ULP); PolyKAN [Li et al., 2025], which implements fused CUDA kernels for polynomial Kolmogorov–Arnold networks via 2D tiling; and FlashInfer (`https://github.com/flashinfer-ai/flashinfer`), which contains

Table 5: Computational performance comparison (Lorenz-63, depth 2, $N = 16$, 720 parameters, 300 training epochs, batch size 2048). Training time is mean ± std over 3 runs. Inference latency is median of 1,000 timed forward passes (lift → predict → reconstruct) after 100 warm-up passes. Rollout time is a 500-step iterative prediction. All GPU measurements include CUDA synchronization barriers.

Backend	Train Time (s)	Train (pairs/s)	Infer $B=1$ (ms)	Infer $B=4096$ (ms)	Peak Mem (MB)
CPU Taylor	13.5 ± 0.4	446,000	0.83	34.0	—
GPU Taylor	3.2 ± 0.1	1,849,000	2.55	2.65	37
GPU Native	$\mathbf{1.8 \pm 0.3}$	**3,379,000**	**0.82**	**0.90**	37
GPU Compiled	2.4 ± 1.0	2,973,000	0.88	0.96	50

PTX-level wrappers for `ex2.approx` and `lg2.approx` instructions. Our GPU Native backend uses CUDA's built-in math functions, which employ the same mathematical techniques as SLEEF (range reduction plus minimax polynomials).

6.2.3 Measured Results

Table 5 reports training throughput, inference latency, and peak GPU memory for all four backends, measured on an NVIDIA RTX 5070 Ti (16 GB, CUDA 12.8) with PyTorch 2.11.

GPU Taylor is kernel-launch dominated. Moving the Taylor implementation to GPU yields a 4.2× training speedup (13.5 → 3.2 s), but inference latency is *worse* than CPU at small batch sizes (2.55 vs. 0.83 ms at $B = 1$) and nearly constant across batch sizes (2.55–2.65 ms). This is because the Horner-form evaluation dispatches $d \times (M_e + M_l) \approx 2 \times 28 = 56$ sequential kernel launches per forward pass — each launch incurs ∼5–10 μs overhead, which dominates at small batch sizes.

GPU Native eliminates Taylor overhead. Replacing the Python-level Horner loops with CUDA's built-in `expf`/`logf` reduces the EML operation to two kernel calls per tree level (one for exp, one for ln), yielding the fastest overall backend: 7.5× training speedup over CPU (13.5 → 1.8 s) and 1.8× over GPU Taylor. Inference at $B = 4096$ drops from 2.65 ms (GPU Taylor) to 0.90 ms — a 2.9× improvement.

GPU Compiled matches GPU Native after warm-up. The `torch.compile` backend (which fuses exp-minus-log into a single Triton kernel) achieves nearly identical steady-state performance to GPU Native: 0.96 ms at $B = 4096$ vs. 0.90 ms. However, the first training run incurs a ∼2 s compilation overhead (3.8 s vs. 1.6 s for subsequent runs), reflected in the higher standard deviation. Peak memory is 50 MB vs. 37 MB due to Triton's code cache.

CPU wins at $B = 1$ inference. For single-sample inference ($B = 1$), CPU Taylor (0.83 ms) outperforms GPU Taylor (2.55 ms) due to kernel launch and host–device transfer overhead. GPU Native (0.82 ms) matches CPU because the reduced kernel count amortizes the launch cost. This has practical implications for real-time control applications where inference is one-sample-at-a-time.

6.2.4 Batch Size Scaling

Table 6 shows inference latency across batch sizes for all four backends.

Table 6: Inference latency (ms) vs. batch size. CPU scales linearly; GPU Taylor is flat (kernel-launch dominated); GPU Native and Compiled are nearly flat with marginal increase at large batches. Values are medians of 1,000 timed passes after 100 warm-up passes.

Backend	$B=1$	$B=64$	$B=256$	$B=1024$	$B=4096$
CPU Taylor	0.83	1.51	3.28	9.79	33.97
GPU Taylor	2.55	2.63	2.63	2.64	2.65
GPU Native	**0.82**	**0.90**	**0.90**	**0.90**	**0.90**
GPU Compiled	0.88	0.97	0.97	0.98	0.96

The CPU backend's linear scaling ($0.83 \rightarrow 34$ ms from $B = 1$ to $B = 4096$) reflects sequential processing of batch elements. GPU backends process all batch elements in parallel, so latency is roughly constant until the GPU's streaming multiprocessors saturate. The crossover where GPU Native overtakes CPU occurs between $B = 1$ and $B = 64$, making GPU the preferred backend for any practical batch size in online prediction or multi-trajectory training.

6.2.5 Implications for CUDA Kernel Development

The 2.9× inference speedup from GPU Taylor to GPU Native demonstrates that the Horner-form Taylor loops are the primary bottleneck on GPU, not the arithmetic itself. A hand-written fused CUDA kernel (as described in the existing SLEEF and PolyKAN literature) would need to beat GPU Native's 0.90 ms at $B = 4096$, which is already close to the hardware floor for 16 dictionary evaluations over a depth-2 tree. The marginal benefit of custom kernels is therefore limited for shallow trees but may become significant at depth ≥ 4 where the number of sequential kernel launches grows exponentially.

6.3 Limitations and Future Work

The progressive depth analysis (Section 5.3) revealed that naively increasing tree depth degrades both E1 and E2, contradicting the expectation that a richer function space should improve approximation quality. This identifies several concrete limitations and motivates targeted experiments.

Limitation 1: Gradient attenuation in deep EML trees. The formula collapse at depth ≥ 3 (Table 3) indicates that gradient signal fails to propagate through nested clamped exponentials. Each EML node applies $\exp(\text{clamp}(x))$, whose gradient is $\exp(\text{clamp}(x))$ when $|x| < C$ and zero otherwise. At depth d, the backward pass multiplies d such terms, creating either exponential gradient magnitudes (when activations are large) or vanishing gradients (when the clamp saturates). The tight clamp required for numerical stability ($C = 3$–5) exacerbates this by restricting the active gradient region.

Limitation 2: Structural E2 gap. The 65-point E2 gap to EDMD-Poly is not a training failure but a fundamental expressiveness mismatch: depth-2 EML trees cannot represent polynomials, and the exponential/logarithmic observables they produce have the wrong spectral signature for long-term Lorenz statistics.

Limitation 3: Initialization sensitivity. All runs used random initialization. The depth-2 success suggests that shallow trees benefit from a relatively smooth loss landscape where Gumbel-

softmax exploration suffices. Deeper trees likely have many more local minima, making random initialization inadequate.

6.3.1 Suggested Further Experiments

The following experiments target the specific bottlenecks identified above:

1. **Curriculum depth growth.** Train at depth 2 until convergence (Phase 3 snap), then grow each tree by appending new leaf nodes with random logits and resume training with the routing structure from the shallower tree frozen. This provides a strong initialization for the deeper tree and avoids the cold-start problem that plagues depth-3+ random initialization. The experiment should sweep growth schedules: $2 \to 3$, $2 \to 3 \to 4$, and $2 \to 4$ (skipping depth 3).

2. **Hybrid EML+Polynomial dictionary.** Augment the EML observable dictionary with polynomial features (monomials up to degree 4) to directly address the E2 gap. The combined dictionary $\mathbf{g}(\mathbf{x}) = [\mathrm{EML}_1(\mathbf{x}), \ldots, \mathrm{EML}_N(\mathbf{x}),\ x_0^2, x_0x_1, \ldots]$ would let the Koopman matrix leverage polynomial features for long-term spectral fidelity while retaining EML's symbolic interpretability for the learned components. This is straightforward to implement within the PySINDy `ConcatLibrary` framework.

3. **Gradient-aware clamping.** Replace hard clamping with a smooth saturation function (e.g., $C \cdot \tanh(x/C)$) that preserves nonzero gradients everywhere. This should reduce gradient attenuation at depth 3+ while still bounding exp inputs. Alternatively, apply gradient clipping *per tree level* rather than globally, allowing the optimizer to maintain signal through deeper layers.

4. **EDMD bootstrap initialization.** Run EDMD-Poly first, identify the top-k polynomial observables by Koopman reconstruction error, then initialize EML tree routing to approximate those polynomials (using known EML representations of monomials). This warm-starts the optimizer near a good solution, which may be especially beneficial at depth ≥ 5 where polynomial representations become expressible.

5. **Spectral loss augmentation.** Add an explicit spectral matching term to the training loss: $\mathcal{L}_{\mathrm{spec}} = \| \log P_{\mathrm{pred}}(f) - \log P_{\mathrm{truth}}(f) \|^2$ over the first k frequency bins. This directly optimizes for E2 during training rather than relying on the one-step prediction loss to implicitly capture long-term statistics.

6. **Complex-valued EML for oscillatory systems.** As demonstrated in Section 6.4, a mixed real/complex dictionary recovers most short-term accuracy while adding oscillatory observables. Further work could explore adaptive real/complex tree ratios, complex-aware spectral losses, and applications to systems with dominant oscillatory modes (e.g., coupled oscillators, fluid flows).

7. **EML-KAN hybrid architectures.** Replace KAN's B-spline activation functions [Liu et al., 2025] with EML nodes, combining KAN's universal approximation structure with EML's symbolic snap. The Kolmogorov–Arnold representation theorem guarantees that any continuous function can be decomposed into univariate functions composed with addition; if each univariate function is an EML tree, the entire network snaps to a symbolic expression.

Table 7: Complex EML comparison on Lorenz-63 CTF metrics (depth 2, $N = 16$). Initial experiments (top) show depth collapse; four depth-collapse fixes are evaluated below. D2 = number of trees retaining depth-2 composition after snapping.

Config	E1	E2	RMSE	Steps	D2	Params
Initial experiments (v1)						
Real baseline	**7.92**	−21.71	**0.886**	**142**	16	720
i + ix original	−1.52	−18.32	0.977	111	0	1,152
With depth-collapse fixes (v2)						
+ child-logit bias	−0.00	−215	0.962	126	16	1,152
+ warm-start	−0.05	−57.8	0.963	126	16	1,152
+ mixed dict	1.89	−22.9	0.944	135	11	960
+ slow anneal	−2.53	−58.1	0.987	107	0	1,152
+ all combined	2.22	**−18.9**	0.941	137	16	960

6.4 Complex EML Primitives

The Lorenz system possesses complex Koopman eigenvalues corresponding to oscillatory dynamics on the attractor. To test whether the EML grammar benefits from access to trigonometric functions, we extend the routing candidate set with imaginary-valued primitives and compare three configurations:

1. **Real baseline** — candidates $\{1, x_0, x_1, x_2\}$ (original grammar).
2. **i-only** — candidates $\{1, i, x_0, x_1, x_2\}$; adds the imaginary unit as a selectable leaf.
3. **i + ix** — candidates $\{1, i, x_0, x_1, x_2, ix_0, ix_1, ix_2\}$; adds both the imaginary unit and imaginary-scaled state variables.

In complex mode the Koopman matrix K is complex-valued, and reconstruction uses $\hat{x} = C_{\text{re}}\,\Re(g) + C_{\text{im}}\,\Im(g)$ to map complex observables back to real state space. The prediction loss becomes $\mathcal{L}_{\text{pred}} = \frac{1}{N}\sum_i |g^{\text{pred}}_{k+1,i} - g^{\text{true}}_{k+1,i}|^2$, where $|\cdot|$ is the complex modulus.

Depth collapse under complex grammar expansion. Initial experiments revealed that complex-mode trees collapsed to depth-1 formulas (D2 = 0/16), losing the nested composition that powered the real baseline. The root cause is *candidate dilution*: with 8 leaf candidates (vs. 4 for real), the Gumbel-softmax probability mass for the child-node slot is halved during soft exploration, causing the optimizer to settle into depth-1 local minima before annealing can harden the child selection.

Four depth-collapse fixes. We tested: (1) a child-logit initialization bias of +2.0 on non-leaf levels; (2) warm-start training (800 real epochs, then expand logits); (3) a mixed dictionary with half real, half complex trees; and (4) extended exploration phase (75% of training at high τ). Table 7 shows that the child-logit bias (1) and warm-start (2) both recover depth-2 composition (D2 = 16) but at the cost of catastrophically worse E2, because they force premature composition of poorly-routed complex leaf nodes. Slow annealing (4) alone does not prevent collapse.

Mixed dictionary is the recommended approach. The mixed dictionary (3) uniquely preserves short-term accuracy (E1 = 1.89, 135 valid steps) while adding complex observables. The real half maintains depth-2 composition with the same $\mathrm{eml}(\mathrm{eml}(1, x_i), x_j)$ patterns found in the pure-real baseline; the complex half discovers oscillatory forms such as $\mathrm{eml}(i, ix_1)$ and $\mathrm{eml}(i, \mathrm{eml}(i, x_0))$. The combined configuration (all fixes, E1 = 2.22, E2 = −18.9) achieves the best balance among depth-2 complex models.

Depth-2 complex trees do not improve E2 over depth-1. A surprising finding is that the v1 depth-1 complex formulas (E2 = −18.3) outperform all v2 depth-2 complex configurations. Nested complex exponentials such as $\exp(\exp(ix))$ produce ill-conditioned spectral signatures, whereas shallow $\exp(ix_j) - \ln(x_k)$ captures the attractor's oscillatory structure more faithfully.

7 Conclusion

We have presented a framework for learning Koopman operator representations using EML observable dictionaries — parameterized binary trees of the universal $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ operator. The completeness of EML over elementary functions provides a principled, gradient-trainable search space that is guaranteed to contain Koopman eigenfunctions whenever they are elementary. GPU-optimized Horner-form Taylor primitives eliminate transcendental function calls, and Gumbel-softmax annealing with three-phase training produces exact symbolic observables upon convergence.

A complete evaluation across all 12 CTF metrics on the Lorenz-63 system yielded an average score of −3.71, placing EML-Koopman within one point of SINDy (−3.00) and 16 points above PyKoopman (−20.11) — competitive with the best interpretable methods while using only 720 parameters. Beyond forecasting, the full CTF evaluation revealed an unexpected strength: the Koopman lift→reconstruct pipeline provides effective denoising (E3 = 33.68, E5 = 30.65), and the compact model size confers natural robustness under data scarcity (E7 = 7.77 with 10% training data). Every discovered observable is a closed-form symbolic expression, yielding interpretable Koopman eigenfunctions that black-box methods cannot produce.

A systematic depth sweep across 11 configurations (depths 2–5, 16–32 observables) revealed that deeper trees do not improve performance from random initialization: E1 degrades from 7.70 (depth 2) to as low as 1.86 (depth 3), and E2 remains negative at all depths. Inspection of snapped formulas shows that depth-3+ trees collapse to root-only expressions, indicating gradient attenuation through nested clamped exponentials. The persistent negative long-term spectral scores (E2, E4, E6, E8) across all conditions reflect a structural mismatch: depth-2 EML trees cannot express polynomials, whose spectral signatures dominate Lorenz long-term statistics. Parametric generalization (E11, E12) remains weak without explicit parameter conditioning.

Extending the grammar with complex-valued primitives (Section 6.4) revealed a depth-collapse phenomenon: the expanded candidate set dilutes Gumbel-softmax routing, causing trees to flatten to depth 1. A mixed dictionary (half real, half complex trees) recovers E1 to 1.89 while gaining access to oscillatory observables, and the combined-fixes configuration achieves the best complex E2 (−18.9) with full depth-2 composition.

These findings point to four high-priority directions: (1) curriculum depth growth, where shallow-trained trees provide warm-start initialization for deeper structures; (2) hybrid EML+polynomial dictionaries that combine symbolic interpretability with spectral fidelity; (3) gradient-aware clamping via smooth saturation to maintain signal through deep trees; and (4) parameter-conditioned Koopman matrices $\mathbf{K}(\rho)$ to enable generalization across bifurcation parameters. A systematic com-

putational benchmark (Section 6.2) on an RTX 5070 Ti demonstrated 7.5× training speedup from CPU to GPU with native math functions, and identified kernel launch overhead from the Python-level Horner loops as the primary GPU bottleneck — resolved by switching to CUDA's built-in `exp`/`log` or `torch.compile` fusion.

Joint real-complex training that combines real-grammar short-term accuracy with complex-grammar spectral fidelity, EML-KAN hybrid architectures, and deeper kernel fusion for depth-≥ 4 trees remain promising longer-term directions.

References

Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. doi: 10.1073/pnas.1517384113.

Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman theory for dynamical systems. *SIAM Review*, 64(2):229–340, 2022. doi: 10.1137/21M1401243.

Marko Budišić, Ryan Mohr, and Igor Mezić. Applied Koopmanism. *Chaos*, 22(4):047510, 2012. doi: 10.1063/1.4772195.

Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. 2023.

Brian M. de Silva, Kathleen Champion, Markus Quade, Jean-Christophe Loiseau, J. Nathan Kutz, and Steven L. Brunton. PySINDy: a Python package for the sparse identification of nonlinear dynamical systems from data. *Journal of Open Source Software*, 5(49):2104, 2020. doi: 10.21105/joss.02104.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. In *ICLR 2017*, 2017.

Alan A. Kaptanoglu, Brian M. de Silva, Urban Fasel, Kadierdan Kaheman, Jared L. Callaham, Charles B. Delahunt, Kathleen Champion, Jean-Christophe Loiseau, J. Nathan Kutz, and Steven L. Brunton. PySINDy: a comprehensive Python package for robust sparse system identification. *Journal of Open Source Software*, 7(69):3994, 2022. doi: 10.21105/joss.03994.

Bernard O. Koopman. Hamiltonian systems and transformations in Hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931. doi: 10.1073/pnas.17.5.315.

Yuehan Li, Jiahao Li, Yichen Chen, Zhanpeng Chu, Yuxuan Liang, and Junfeng Huang. PolyKAN: Efficient fused GPU operators for polynomial Kolmogorov–Arnold network variants. 2025.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. KAN: Kolmogorov-Arnold networks. In *ICLR 2025*, 2025.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9:4950, 2018. doi: 10.1038/s41467-018-07210-0.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

Andrzej Odrzywołek. All elementary functions from a single operator. 2026.

Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. doi: 10.1017/S0022112010001217.

Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009. doi: 10.1126/science.1165893.

Henry M. Sheffer. A set of five independent postulates for Boolean algebras, with application to logical constants. *Transactions of the American Mathematical Society*, 14(4):481–488, 1913. doi: 10.1090/S0002-9947-1913-1500960-1.

Naoki Shibata. SLEEF: SIMD library for evaluating elementary functions. `https://sleef.org`, 2025. Version 3.9.

Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning Koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, 2014. doi: 10.3934/jcd.2014.1.391.

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015a. doi: 10.1007/s00332-015-9258-5.

Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven Koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, 2015b. doi: 10.3934/jcd.2015005.

Patrick M. Wyder, Jonathan Goldfeder, Alexander Yermakov, Yifan Zhao, Serina Riva, James Williams, Daniel Zoro, Amber S. Rude, Matteo Tomasetto, Joseph Germany, Joseph Bakarji, Georg Maierhofer, Miles Cranmer, and J. Nathan Kutz. Common task framework for a critical evaluation of scientific machine learning algorithms. In *NeurIPS 2025 Datasets and Benchmarks Track*, 2025.